MySQL

1️⃣ SQL Basics (Must Know)

- What is DB, RDBMS
- Tables, Rows, Columns
- Data Types (INT, VARCHAR, DATE)
- `CREATE`, `INSERT`, `UPDATE`, `DELETE`
- `SELECT`, `WHERE`, `ORDER BY`

2️⃣ SQL Constraints (VERY IMPORTANT)

- `PRIMARY KEY`
- `FOREIGN KEY`
- `UNIQUE`
- `NOT NULL`
- `CHECK`

3️⃣ Joins (🔥 Interview Favorite)

- `INNER JOIN`
- `LEFT JOIN`
- `RIGHT JOIN`
- `FULL JOIN`
- Self Join

4️⃣ Advanced SQL (PRODUCT COMPANY LEVEL)

- `GROUP BY`, `HAVING`
- Subqueries
- Correlated Subqueries

- Indexes (why & when)
- Normalization (1NF, 2NF, 3NF)
- Transactions
- ACID properties

What is Database?

Database is a collection of interrelated data.

What is DBMS?

DBMS (Database Management System) is software used to create, manage, and organize
databases.

What is RDBMS?

● RDBMS (Relational Database Management System) - is a DBMS based on the concept of tables (also called relations).
● Data is organized into tables (also known as relations) with rows (records) and columns (attributes).
● Eg - MySQL, PostgreSQL, Oracle etc.

What is SQL?

SQL (Structured Query Language) is used to **store, retrieve, and manipulate data** in relational databases.

Core SQL Commands (CRUD)

Operation	Command
Create	`INSERT`
Read	`SELECT`
Update	`UPDATE`
Delete	`DELETE`

SQL v/s MySQL
SQL is a language used to perform CRUD operations in Relational DB, while MySQL is a
RDBMS that uses SQL.

Example Table

```
CREATE TABLE users (
  idINT,
  name VARCHAR(50),
  email VARCHAR(100),
  age INT
);
```

Insert Data

```
INSERT INTO users VALUES (1,'Suraj','suraj@gmail.com',22);
```

Fetch Data

```
SELECT*FROM users;
SELECT name, email FROM users WHERE age>20;
```

📌 **Interview Tip**

❓ *Difference between DELETE & TRUNCATE?*

- DELETE → row by row, rollback possible
- TRUNCATE → removes all rows, faster, no rollback

SQL Data Types

STRING DATATYPES

DATATYPE	DESCRIPTION	USAGE
CHAR	string(0-255), can store characters of fixed length	CHAR(50)
VARCHAR	string(0-255), can store characters up to given length	VARCHAR(50)
BLOB	string(0-65535), can store binary large object	BLOB(1000)

NUMERIC DATATYPES

DATATYPE	DESCRIPTION	USAGE
INT	integer(-2,147,483,648 to 2,147,483,647)	INT
TINYINT	integer(-128 to 127)	TINYINT
BIGINT	integer(-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807)	BIGINT
BIT	can store x-bit values. x can range from 1 to 64	BIT(2)
FLOAT	Decimal number - with precision to 23 digits	FLOAT
DOUBLE	Decimal number - with 24 to 53 digits	DOUBLE

BOOLEAN DATATYPE

DATATYPE	DESCRIPTION	USAGE
BOOLEAN	Boolean values 0 or 1	BOOLEAN

DATE & TIME DATATYPES

DATATYPE	DESCRIPTION	USAGE
DATE	date in format of YYYY-MM-DD ranging from 1000-01-01 to 9999-12-31	DATE
TIME	HH:MM:SS	TIME
YEAR	year in 4 digits format ranging from 1901 to 2155	YEAR

Types of SQL Commands:

1. DQL (Data Query Language) : Used to retrieve data from databases. (SELECT)
2. DDL (Data Definition Language) : Used to create, alter, and delete database objects
   like tables, indexes, etc. (CREATE, DROP, ALTER, RENAME, TRUNCATE)
3. DML (Data Manipulation Language): Used to modify the database. (INSERT, UPDATE, DELETE)
4. DCL (Data Control Language): Used to grant & revoke permissions. (GRANT, REVOKE)

5. TCL (Transaction Control Language): Used to manage transactions. (COMMIT, ROLLBACK, START TRANSACTIONS, SAVEPOINT)

DDL Commands (Data Definition Language)

DDL commands are used to define and modify database structure/schema.

🔷 Main DDL Commands

1️⃣ CREATE

Used to **create** database objects.

➤ Create Database

```
CREATE DATABASE company;
```

➤ Create Table

```
CREATE TABLE employees (
    emp_idINTPRIMARY KEY,
    nameVARCHAR(50),
    salaryDECIMAL(10,2),
    departmentVARCHAR(30)
);
```

2️⃣ DROP

Used to **delete database objects permanently**.

➤ Drop Table

```
DROP TABLE employees;
```

➤ Drop Database

```
DROP DATABASE company;
```

⚠️ **Warning:** Data + structure both are lost.

3️⃣ ALTER

Used to **modify an existing table structure**.

➤ Add Column

```
ALTER TABLE employees
ADD emailVARCHAR(50);
```

➤ Modify Column

```
ALTER TABLE employees
MODIFY salaryDECIMAL(12,2);
```

➤ Drop Column

```
ALTER TABLE employees
DROPCOLUMN email;
```

4️⃣ TRUNCATE

Used to **remove all records** from a table but **keeps the structure**.

```
TRUNCATETABLE employees;
```

✔️ Faster than `DELETE`

❌ Cannot be rolled back

❌ No `WHERE` clause

5️⃣ `RENAME`

Used to **rename a table**.

```
RENAMETABLE employeesTO staff;
```

(Or in some DBs)

```
ALTER TABLE employees RENAME TO staff;
```

🔷 DDL Commands Summary Table

Command	Purpose
CREATE	Create database/table
DROP	Delete database/table
ALTER	Modify table structure
TRUNCATE	Delete all records only
RENAME	Rename database object

📌 The **main and most important DQL command is** `SELECT`.

🔷 What is DQL?

- Used to **fetch data**
- Works only on existing tables
- Does **not change** table structure or data
- Most frequently used in real projects & interviews

🔷 Main DQL Command

1️⃣ `SELECT`

Used to retrieve data from one or more tables.

➤ Select all columns

```
SELECT*FROM employees;
```

➤ Select specific columns

```
SELECT name, salary FROM employees;
```

🔷 DQL with `WHERE` (Filtering)

```
SELECT*
FROM employees
WHERE salary>50000;
```

✔️ Filters rows

✔️ Works with conditions

🔷 DQL with Logical Operators

```
SELECT*
FROM employees
WHERE department='IT' AND salary>60000;
```

Operators:

- `AND`
- `OR`
- `NOT`

🔷 DQL with Comparison Operators

Operator	Meaning
=	Equal
!= / <>	Not equal
>, <	Greater / Less
>=, <=	Greater/Less equal

🔷 DQL with `LIKE` (Pattern Matching)

```
SELECT *
FROM employees
WHERE name LIKE 'S%';
```

✔️ Starts with **S**

```
SELECT *
FROM employees
WHERE nameLIKE '%ar%';
```

✔️ Contains **ar**

🔷 DQL with `IN` & `BETWEEN`

➤ `IN`

```
SELECT*
FROM employees
WHERE department IN ('IT','HR');
```

➤ `BETWEEN`

```
SELECT*
FROM employees
WHERE salary BETWEEN 40000 AND 80000;
```

🔷 DQL with `ORDER BY`

```
SELECT*
FROM employees
ORDERBY salary DESC;
```

✔️ `ASC` → default

✔️ `DESC` → descending

🔷 DQL with `LIMIT` / `OFFSET`

```
SELECT*
FROM employees
```

```
LIMIT 5;
```

```
SELECT*
FROM employees
LIMIT 5 OFFSET 10;
```

(MySQL / PostgreSQL)

🔷 DQL with Aggregate Functions

Function	Purpose
COUNT()	Number of rows
SUM()	Total
AVG()	Average
MIN()	Minimum
MAX()	Maximum

```
SELECT AVG(salary) FROM employees;
```

🔷 DQL with `GROUP BY`

```
SELECT department, AVG(salary)
FROM employees
GROUPBY department;
```

🔷 `HAVING` vs `WHERE` (Interview ⭐)

```
SELECT department,COUNT(*)
FROM employees
GROUPBY department
HAVING COUNT(*) > 5;
```

WHERE	HAVING
Filters rows	Filters groups
Used before GROUP BY	Used after GROUP BY

🔷 DQL with `DISTINCT`

```
SELECT DISTINCT department FROM employees;
```

✔️ Removes duplicates

🔷 What is DML?

- Used to **manipulate records (rows)**
- Changes **table data**
- Can be **rolled back** (when used with transactions)
- Most used in **real applications & backend APIs**

🔷 Main DML Commands

1️⃣ `INSERT`

Used to **add new records** into a table.

➤ Insert single row

```
INSERT INTO employees (emp_id, name, salary, department)
VALUES (1,'Suraj',60000,'IT');
```

➤ Insert multiple rows

```
INSERT INTO employeesVALUES
(2,'Amit',55000,'HR'),
(3,'Riya',70000,'Finance');
```

2️⃣ `UPDATE`

Used to **modify existing records**.

➤ Update specific rows

```
UPDATE employees
SET salary=65000
WHERE emp_id=1;
```

⚠️ **Always use** `WHERE` to avoid updating all rows.

3️⃣ `DELETE`

Used to **remove records** from a table.

➤ Delete specific rows

```
DELETE FROM employees
WHERE department='HR';
```

➤ Delete all rows

```
DELETE FROM employees;
```

🔷 `DELETE` vs `TRUNCATE` (Interview ⭐)

DELETE	TRUNCATE
DML	DDL
Can use WHERE	No WHERE
Can rollback	Cannot rollback
Slower	Faster

🔷 DML with `SELECT` (INSERT FROM SELECT)

```
INSERT INTO backup_employees
SELECT*FROM employees
WHERE department='IT';
```

🔷 DML with Conditions

```
UPDATE employees
SET department='Tech'
WHERE department='IT';
```

🔷 Transactions in DML (Important ⭐)

```
START TRANSACTION;
```

```
UPDATE employeesSET salary= salary+5000WHERE department='IT';
DELETEFROM employeesWHERE emp_id=3;

ROLLBACK;-- undo changes
-- COMMIT; -- save changes
```

✔️ Makes DML safe

✔️ Used heavily in backend systems

🔷 DML Interview One-Liners ⭐

- **DML modifies table data**
- **INSERT, UPDATE, DELETE are DML commands**
- **DELETE can be rolled back, TRUNCATE cannot**
- **DML supports transactions**

🔷 Real-World Backend Usage

- User registration → `INSERT`
- Profile update → `UPDATE`
- Account deletion → `DELETE`
- Bulk operations → `INSERT INTO SELECT`

🔷 SQL Command Categories (Quick Recap)

Category	Commands
DDL	CREATE, ALTER, DROP
DQL	SELECT
DML	INSERT, UPDATE, DELETE
TCL	COMMIT, ROLLBACK
DCL	GRANT, REVOKE

🔷 What is DCL?

- Manages **user permissions**
- Controls **security & authorization**
- Used mainly by **DBAs & backend admins**
- Very important for **enterprise & MNC projects**

📌 It defines **who can do what** in a database.

🔷 Main DCL Commands

1️⃣ GRANT

Used to **give permissions** to users or roles.

➤ Grant permission on a table

```
GRANT SELECT,INSERT ON employees TO user1;
```

➤ Grant all permissions

```
GRANT ALL ON employees TO admin_user;
```

2️⃣ REVOKE

Used to **remove permissions**.

```
REVOKE INSERT ON employees FROM user1;
```

🔷 Role-Based Access Control (RBAC)

```
CREATE ROLE manager;

GRANT SELECT,UPDATE ON employees TO manager;
GRANT manager TO user2;
```

✔️ Cleaner

✔️ Scalable

✔️ Industry standard

🔷 DCL Interview Points ⭐

- **DCL controls database security**
- **GRANT gives permission, REVOKE removes it**
- **DCL does not affect table data**
- **Used mostly by DBAs**

🔷 TRANSACTION CONTROL LANGUAGE (TCL)

TCL (Transaction Control Language) is used to **manage transactions** in a database.

It ensures **data consistency, integrity, and reliability**.

📌 TCL decides **when changes made by DML are permanently saved or undone**.

🔷 What is a Transaction?

A **transaction** is a logical unit of work that contains **one or more DML operations**.

Example:

```
INSERT INTO orders VALUES (...);
UPDATE accounts SET balance= balance-500;
```

These should either **both succeed or both fail**.

🔷 Main TCL Commands

1️⃣ COMMIT

✔️ Permanently saves all changes made in the current transaction.

```
COMMIT;
```

2️⃣ ROLLBACK

❌ Undoes all changes made since the last `COMMIT`.

```
ROLLBACK;
```

3️⃣ SAVEPOINT

📍 Creates a point inside a transaction to roll back partially.

```
SAVEPOINT sp1;
```

```
ROLLBACK TO sp1;
```

4️⃣ SET TRANSACTION

Used to set **transaction properties** (isolation level).

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

🔷 TCL Example (Complete Flow)

```
START TRANSACTION;

INSERT INTO employees VALUES (10,'Ravi',60000,'IT');
SAVEPOINT sp1;

UPDATE employees SET salary=70000WHERE emp_id=10;
ROLLBACK TO sp1;

COMMIT;
```

✔️ Insert saved

✔️ Salary update undone

🔷 ACID Properties (Very Important ⭐)

Property	Meaning
Atomicity	All or nothing
Consistency	Valid state
Isolation	Transactions don't interfere
Durability	Data persists after commit

🔷 When TCL is Used?

- Banking transactions 💳
- E-commerce payments 🛒
- Reservation systems ✈️
- Backend APIs (Spring Boot)

✅ Final Interview Answer (Perfect ⭐)

> Transaction Control Language (TCL) manages database transactions using COMMIT, ROLLBACK, and SAVEPOINT to ensure ACID properties.

🔷 Why Constraints are Used?

- Prevent duplicate values
- Maintain data consistency
- Enforce relationships between tables
- Avoid invalid or NULL data

🔷 Types of SQL Constraints

1️⃣ NOT NULL

Ensures a column **cannot have NULL values**.

```
CREATE TABLE students (
    idINTNOT NULL,
    name VARCHAR(50) NOT NULL
);
```

✔️ Mandatory field

2️⃣ UNIQUE

Ensures **all values are unique** in a column.

```
CREATE TABLE users (
    emailVARCHAR(100)UNIQUE
);
```

✔️ No duplicate values

✔️ Allows only one NULL (DB-dependent)

3️⃣ PRIMARY KEY

- Combination of `NOT NULL` + `UNIQUE`
- Uniquely identifies each row

```
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    name VARCHAR(50)
);
```

✔️ One primary key per table

4️⃣ FOREIGN KEY

Creates a **relationship between two tables**.

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    emp_id INT,
FOREIGN KEY (emp_id) REFERENCES employees(emp_id)
);
```

✔️ Maintains referential integrity

5️⃣ CHECK

Ensures values meet a **specific condition**.

```
CREATE TABLE products (
    price DECIMAL(10,2)CHECK (price>0)
);
```

✔️ Prevents invalid values

6️⃣ `DEFAULT`

Sets a **default value** if none is provided.

```
CREATE TABLE accounts (
    status VARCHAR(20) DEFAULT 'ACTIVE'
);
```

✔️ Automatically assigns value

7️⃣ `AUTO_INCREMENT` / `SERIAL`

Automatically generates unique numbers.

```
CREATE TABLE users (
    idINT AUTO_INCREMENT PRIMARY KEY
);
```

(MySQL)

```
id SERIAL PRIMARY KEY;
```

(PostgreSQL)

🔷 Adding Constraints Using `ALTER`

```
ALTER TABLE employees
ADD CONSTRAINT uq_email UNIQUE (email);
```

🔷 Dropping Constraints

```
ALTER TABLE employees
DROP CONSTRAINT uq_email;
```

(DB-specific syntax)

🔷 Constraint Comparison (Interview ⭐)

Constraint	Purpose
NOT NULL	No empty values
UNIQUE	No duplicates
PRIMARY KEY	Unique + Not Null
FOREIGN KEY	Link tables
CHECK	Condition-based
DEFAULT	Default value

🔷 FOREIGN KEY Options (Advanced ⭐)

```
FOREIGN KEY (emp_id)
REFERENCES employees(emp_id)
ON DELETE CASCADE
ON UPDATE CASCADE;
```

Options:

- `CASCADE`
- `SET NULL`
- `RESTRICT`
- `NO ACTION`

🔷 Real-World Usage

- User ID → `PRIMARY KEY`
- Email → `UNIQUE`
- Salary → `CHECK`
- Status → `DEFAULT`
- Order–Customer → `FOREIGN KEY`

SQL JOINS (Very Important ⭐)

SQL Joins are used to **combine rows from two or more tables** based on a **related column** (usually a foreign key).

📌 Joins are **core for backend development**, reports, and **interview questions**.

🔷 Why Do We Need Joins?

- Data is stored in **multiple normalized tables**
- Joins help **fetch related data together**
- Used in **APIs, dashboards, analytics**

Example:

- `employees` table
- `departments` table

  👉 Join them to get *employee name + department name*

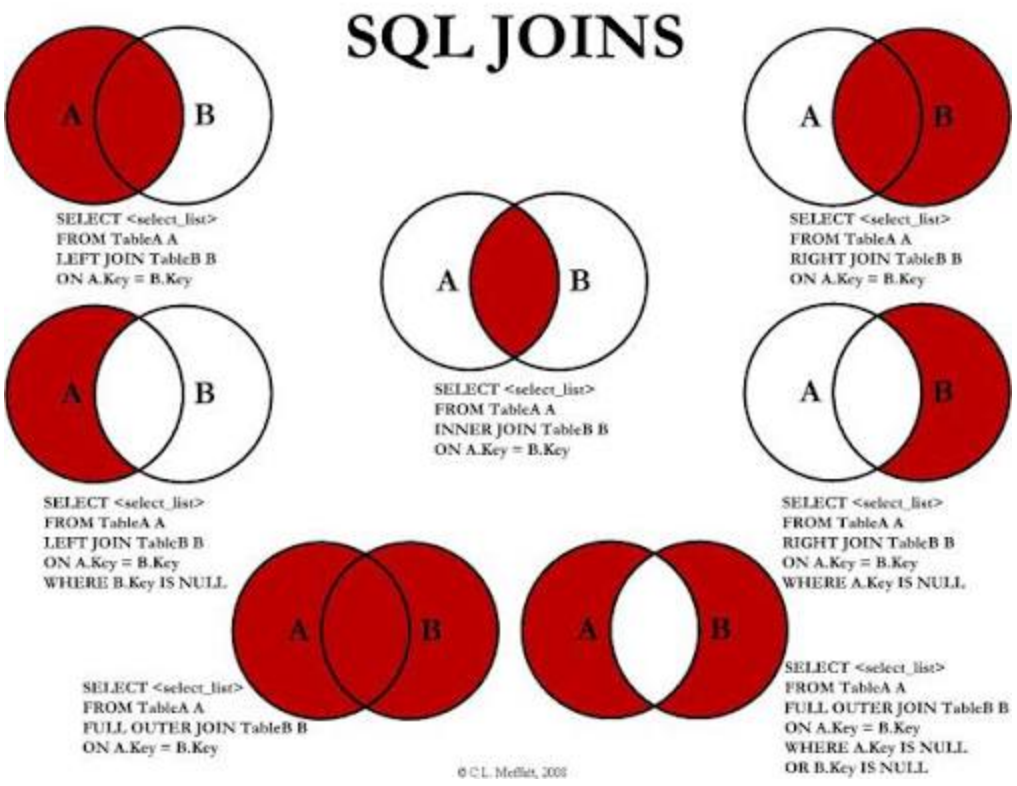
🔷 Tables Used in Examples

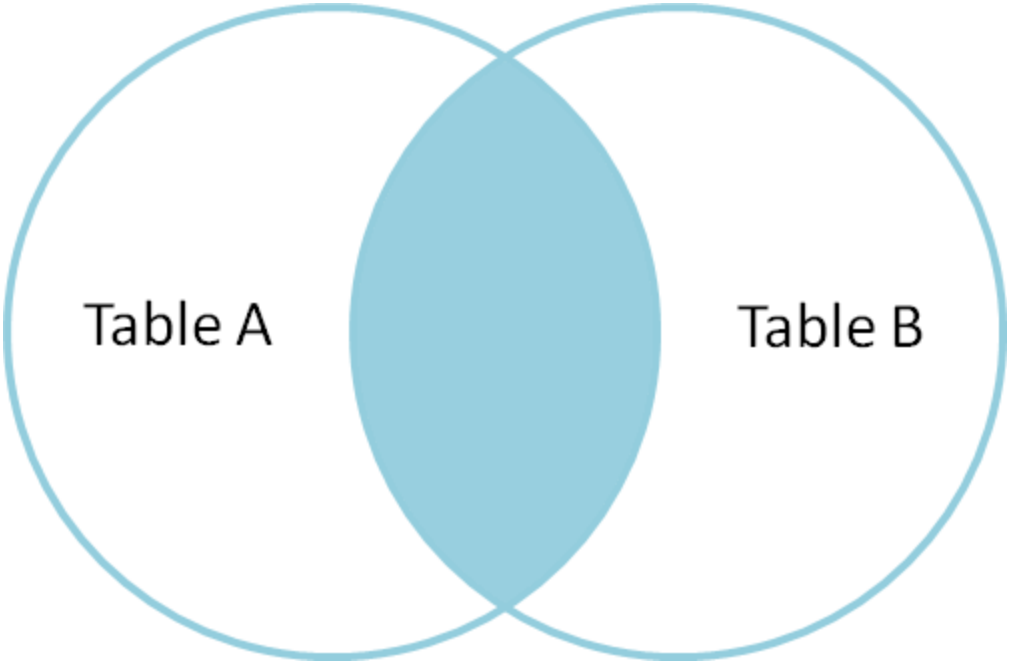
```
employees(emp_id, name, dept_id)
departments(dept_id, dept_name)
```

🔷 Types of SQL Joins

1️⃣ INNER JOIN

✔️ Returns **only matching records** from both tables.





```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id;
```

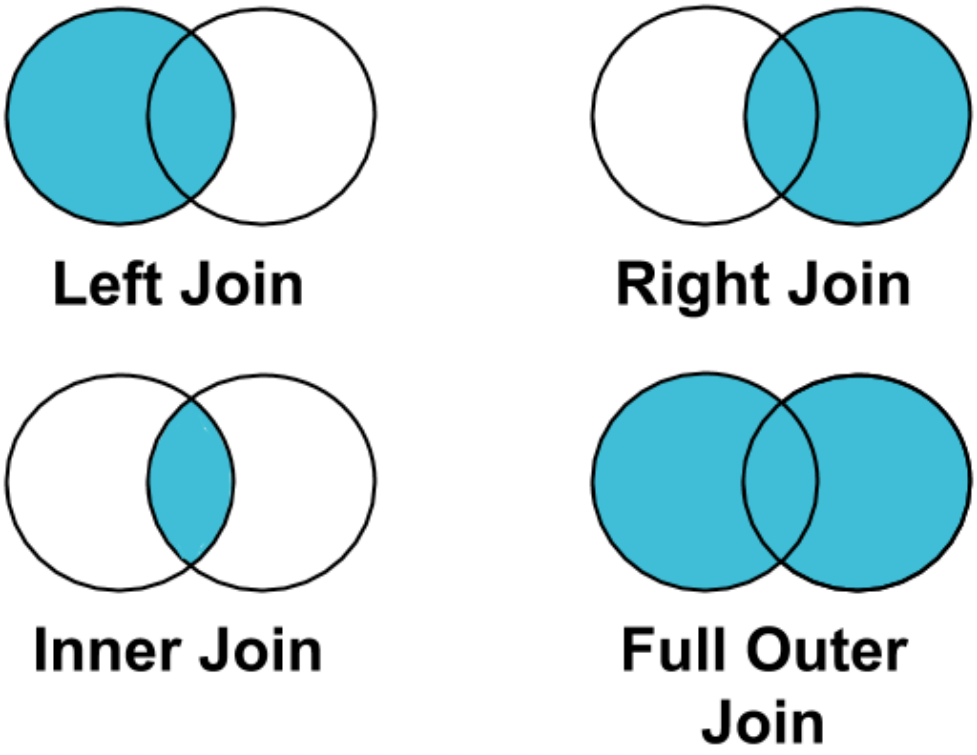
📌 Most commonly used join

2️⃣ LEFT JOIN (LEFT OUTER JOIN)

✔️ Returns **all records from left table**

✔️ Matching records from right table

✔️ Non-matching → `NULL`



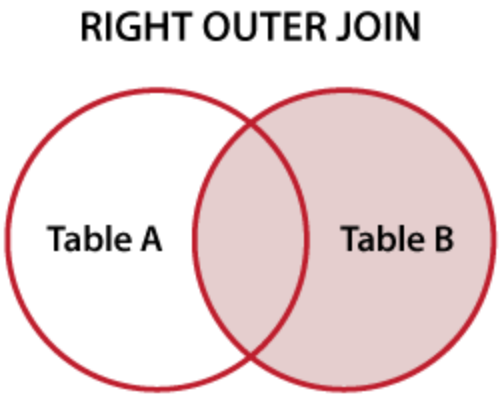
```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id;
```

3️⃣ RIGHT JOIN (RIGHT OUTER JOIN)

✔️ Returns **all records from right table**

✔️ Matching from left

✔️ Non-matching → `NULL`




```
SELECT e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.dept_id;
```

4️⃣ FULL JOIN (FULL OUTER JOIN)

✔️ Returns **all records from both tables**

✔️ Matching + non-matching

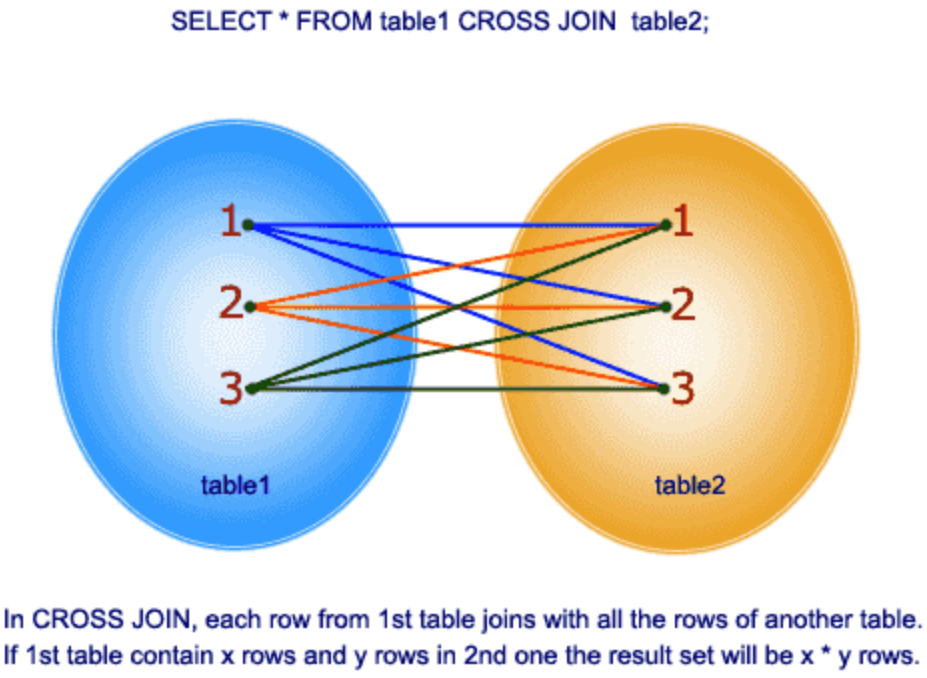
```
SELECT e.name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d
ON e.dept_id= d.dept_id;
```

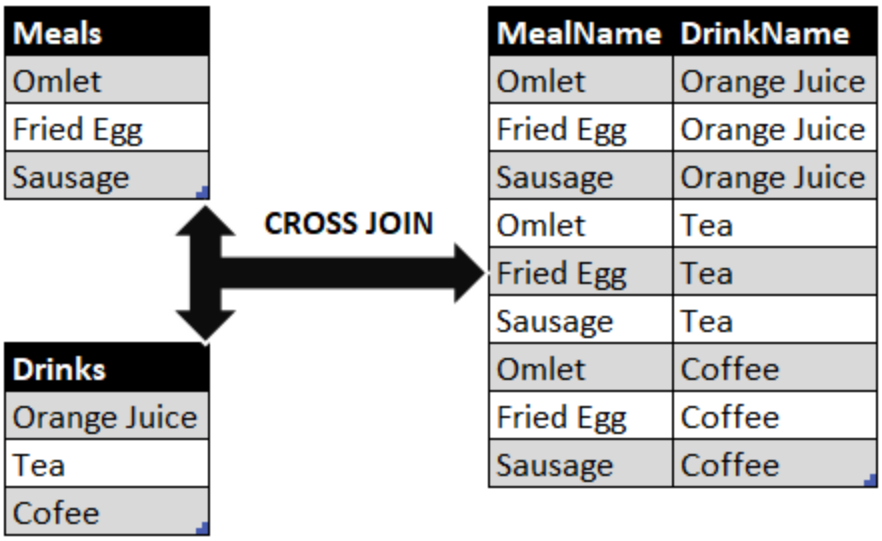
⚠️ Not supported in MySQL (use UNION workaround)

5️⃣ CROSS JOIN

✔️ Returns **Cartesian product**

✔️ Every row from table A × every row from table B






```
SELECT e.name, d.dept_name
FROM employees e
CROSS JOIN departments d;
```

⚠️ Use carefully (huge output)

6️⃣ SELF JOIN

✔️ A table joined with **itself**

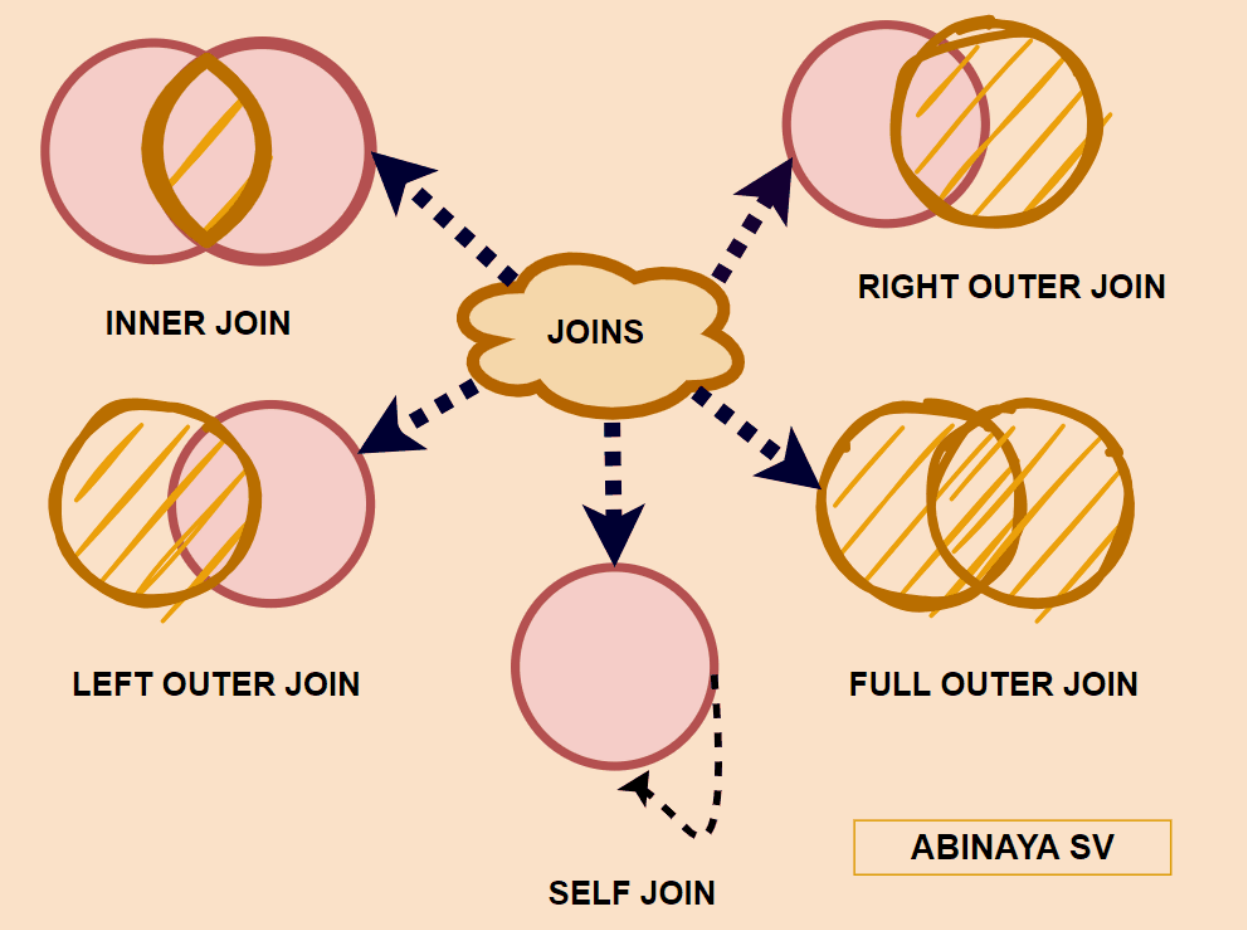
Color	Name	Color
Blue	John	Red
Green	Alex	Blue
Red	Simon	Green

+

Color	Name	Color
Blue	John	Red
Green	Alex	Blue
Red	Simon	Green

=

Name	Secret_Santa
John	Simon
Alex	John
Simon	Alex



```
SELECT e1.nameAS employee, e2.nameAS manager
FROM employees e1
JOIN employees e2
ON e1.manager_id= e2.emp_id;
```

📌 Common in org-hierarchy problems

🔷 Join Comparison (Interview ⭐)

Join Type	Result
INNER	Matching only
LEFT	All left + matching right
RIGHT	All right + matching left
FULL	All records
CROSS	All combinations
SELF	Same table

1️⃣ GROUP BY & HAVING

🔷 GROUP BY

Used to **group rows** that have the same values and apply **aggregate functions**.

```
SELECT department,AVG(salary)
FROM employees
GROUPBY department;
```

✔️ Groups employees by department

✔️ Calculates average salary per department

🔷 HAVING

Used to **filter groups** (NOT rows).

```
SELECT department,COUNT(*)
FROM employees
GROUPBY department
HAVINGCOUNT(*)>5;
```

📌 **Interview rule** ⭐

- `WHERE` → filters rows
- `HAVING` → filters groups

2️⃣ Subqueries (Nested Queries)

A **subquery** is a query inside another query.

🔷 Example: Salary > Average Salary

```
SELECT name
FROM employees
WHERE salary> (
SELECT AVG(salary)FROM employees
);
```

✔️ Inner query runs first

✔️ Outer query uses its result

🔷 Types of Subqueries

- Single-row subquery
- Multi-row subquery (`IN`, `ANY`, `ALL`)
- Subquery in `SELECT`, `FROM`, `WHERE`

3️⃣ Correlated Subqueries (Very Important ⭐)

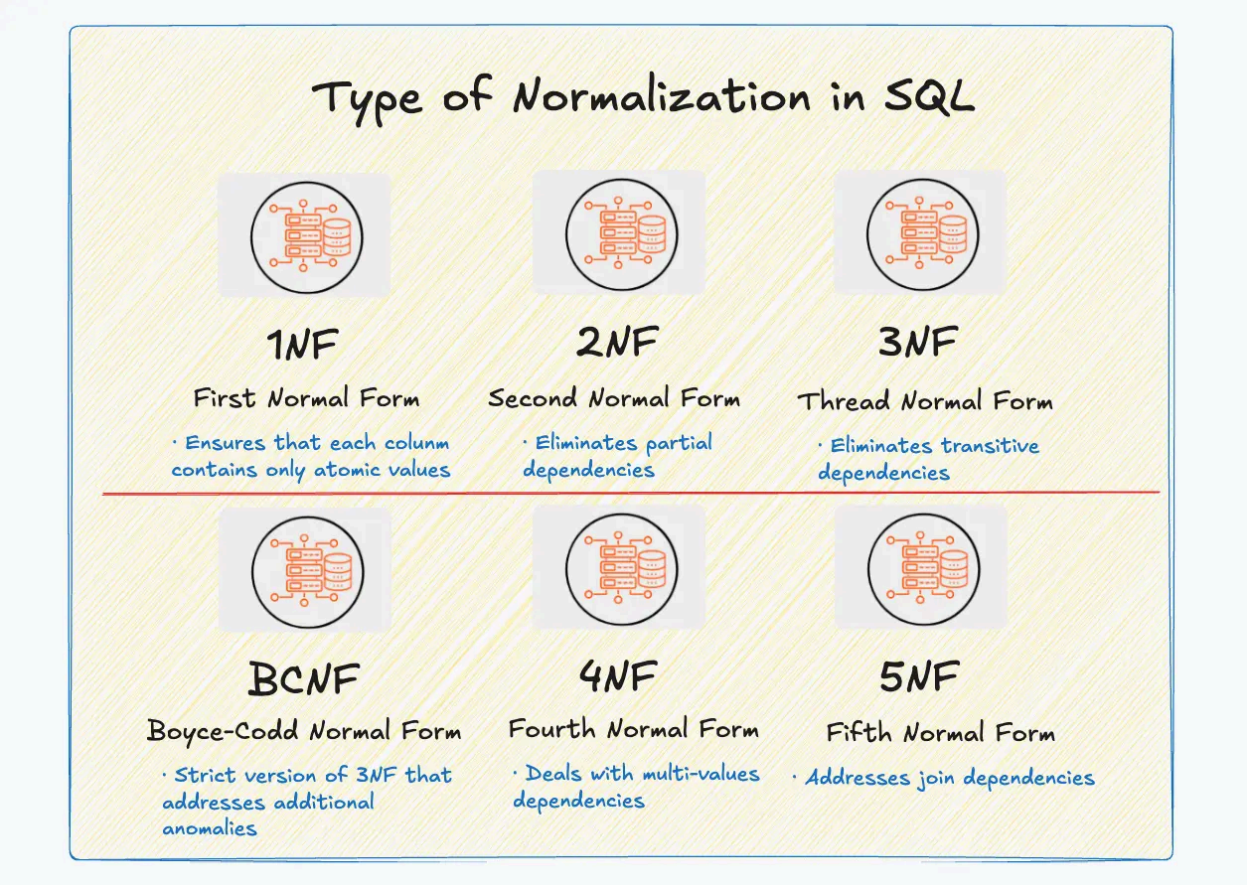
A **correlated subquery** depends on the **outer query** and runs **once per row**.

🔷 Example: Highest salary in each department

```
SELECT e1.name, e1.salary
FROM employees e1
WHERE e1.salary= (
SELECTMAX(e2.salary)
FROM employees e2
WHERE e2.department= e1.department
);
```

5️⃣ Normalization (1NF, 2NF, 3NF)

Normalization removes **redundancy** and improves **data integrity**.



🔷 1NF (First Normal Form)

✔️ Atomic values

✔️ No repeating groups

❌ `skills = Java,SQL`

✔️ `skills = Java` (separate rows)

🔷 2NF

✔️ Must be in 1NF

✔️ No **partial dependency**

📌 Applies to **composite primary keys**

🔷 3NF

✔️ Must be in 2NF

✔️ No **transitive dependency**

📌 Non-key column should not depend on another non-key column

🔷 Interview Summary ⭐

NF	Removes
1NF	Repeating data
2NF	Partial dependency
3NF	Transitive dependency